

-Projektna dokumentacija:

-U ovom dokumentu nalaze se opisi dijagrama, dok ce slike biti u zasebnim .png datotekama.

-Tema: Catan

-Clanovi tima:

1. Luka Bura 218/2019
2. Luka Vukotic 120/2019
3. Stefan Neskovic 286/2021
4. Stefan Drljevic 103/2019

1. Dijagram klasa:

-Dijagram klasa sadrzim sve klase koje se koriste u nasem projektu, kao i njihovo povezivanje. U odnosu na inicijalan dijagram klase, novi dijagram je znatno veci. U prilogu su slike citavog dijagrama klasa.

-Dijagram klasa se koristi se za opšte konceputalno modelovanje strukture aplikacije, kao i za detaljno modelovanje prevođenja modela u programski kod.

```

class Paradox Online Free Edition

Class Player
m_id : int
m_color : Color
m_name : std::string
m_victory_points : int
m_num_of_roads : int
m_num_of_houses : int
m_num_of_cities : int
m_player_resources : std::unordered_map<ResourceType, int>
m_num_of_allowed_houses : int
m_num_of_allowed_cities : int
m_num_of_allowed_roads : int
m_num_of_brick : int
m_num_of_wood : int
m_num_of_wheat : int
m_num_of_stone : int
m_player_color : QColor
m_city_color : QColor

+Player()
+Player(const Player&) = delete
+Player(int, int, std::string, color : QColor, color_c : QColor)
+get_name() const : std::string &
+get_id() const : int
+get_color() const : Color
+get_victory_points() const : int
+get_num_of_houses() const : int
+get_num_of_cities() const : int
+get_num_of_roads() const : int
+get_num_of_wood() const : int
+get_num_of_brick() const : int
+get_num_of_wheat() const : int
+get_num_of_stone() const : int
+get_num_of_allowed_houses() : void
+inc_num_of_wood() : void
+inc_num_of_brick() : void
+inc_num_of_wheat() : void
+inc_num_of_stone() : void
+dec_num_of_wood() : void
+dec_num_of_brick() : void
+dec_num_of_wheat() : void
+dec_num_of_stone() : void
+double_inc_of_wood() : void
+double_inc_of_brick() : void
+double_inc_of_wheat() : void
+double_inc_of_stone() : void
+get_num_of_allowed_houses() const : int
+get_num_of_allowed_cities() const : int
+get_num_of_allowed_roads() const : int
+get_player_color() const : QColor
+get_city_color() const : QColor
+get_id() const : int
+get_resource_type : ResourceType, amount : int, p : Player & : bool
+take_resource_card : ResourceType, amount : int, bank : Bank & : void
+return_resource_card : ResourceType, amount : int, bank : Bank & : void
+get_player_resources() : std::unordered_map<ResourceType, int>
+increase_num_of_houses() : void
+increase_num_of_cities() : void
+increase_num_of_roads() : void
+increase_num_of_victory_points() : void
+decrease_allowed_houses_number() : void
+decrease_allowed_houses_number() : void
+decrease_allowed_cities_number() : void
+decrease_allowed_roads_number() : void
+decrease_victory_points() : void
+compare_players() : const Player* p1, const Player*
+set_name : std::string : void

```

```

Class Road
m_owner : int
m_id : int
m_can_build : false : bool
+Roadowner(int, id : int)
+Road(const Road &) = delete
+get_owner() const : int
+get_id() const : int
+get_can_build() const : bool
+set_owner(owner : int) : void
+set_can_build(value : bool) : void

```

```

Class Node
m_id : int
m_is_house_built : bool
m_is_city_built : bool
m_owner : int
m_can_build : bool
m_can_build_city : false : bool
m_neighbours : QVector<int>
+Node(id : int)
+Node(const Node&) = delete
+get_id() const : int
+get_is_house_built() const : bool
+get_is_city_built() const : bool
+get_owner() const : int
+get_can_build() const : bool
+get_can_build_city() : bool
+set_is_house_built(value : bool) : void
+set_is_city_built(value : bool) : void
+set_owner(owner : int) : void
+set_can_build(value : bool) : void
+set_neighbours(neighbours : QVector<int>) : void
+place_house(owner : int) : void
+delete_city(owner : int) : void

```

```

Class GUI_Road
m_strat : GUI_Node *
m_end : GUI_Node *
m_pen : QPen(QColor blue) : QPen
m_color : static QColor
+GUI_Road(strat : GUI_Node *, end : GUI_Node *)
+GUI_Road()
+boundingRect() const override : QRectF
+paintPainter : QPainter * option : const QStyleOptionGraphicsItem *, widget : QWidget * override : void
+getStrat() : GUI_Node *
+getEnd() : GUI_Node *
+getCenter() : QPen *
+setPen(color : QColor) : void
+needRedraw() : void -> signal

```

```

Class GUI_Node
m_node : Node *
hasRoad : bool
m_brush : QBrush(QColor white) : QBrush
m_is_house_built : false : bool
m_is_end_of_road : false : bool
m_is_ending_road_built : QStyleOptionGraphicsItem *
Q_PROPERTY(Node * node READ node WRITE setNode NOTIFY nodeChanged)
m_color : static QColor
+GUI_Node(Node * node)
+boundingRect() const override : QRectF
+position() const : QRectF
+paintPainter : QPainter * option : const QStyleOptionGraphicsItem *, widget : QWidget * override : void
+setBrush(QBrush brush) : void
+getBrush() : QBrush
+type() const override : int
+setNode(Node * node) : void
+getNode() : Node *
+get_is_house_built() const : bool
+set_is_house_built(value : bool) : void
+check_name_color : QColor() : bool
+get_hasRoad() const : bool
+set_hasRoad(value : bool) : void
+get_is_end_of_road() const : bool
+set_is_end_of_road(value : bool) : void
+nodeChanged() : void -> signal
+needRedraw() : void -> signal
+hoverEnterEvent : QGraphicsSceneHoverEvent * override : void
+hoverLeaveEvent : QGraphicsSceneHoverEvent * override : void

```

```

<<enumerations>>
ResourceType
Wood
Brick
Wheat
Stone

Class Bank
m_bank_resources : std::unordered_map<ResourceType, unsigned>
m_num_of_brick : unsigned
m_num_of_wood : unsigned
m_num_of_wheat : unsigned
m_num_of_stone : unsigned
+Bank()
+Bank(const Bank &) = delete
+get_bank_resources() : std::unordered_map<ResourceType, unsigned> &
+get_num_of_brick_from_bank() const : unsigned
+get_num_of_wood_from_bank() const : unsigned
+get_num_of_wheat_from_bank() const : unsigned
+get_num_of_stone_from_bank() const : unsigned
+add_resource_card : ResourceType, num_cards : int : void
+remove_resource_card : ResourceType, num_cards : int : void

```

```

Class Game
m_players : QVector<Player>
m_player1 : Player *
m_player2 : Player *
m_player3 : Player *
m_player4 : Player *
m_currentPlayer : Player *
m_bank : Bank *
numOfPlayers = 4 : int
m_can_build_house : bool
nextPlayer : Player *
m_win = false : bool
player_turn_counter = 1 : int
+Game()
+Game()
+getPlayer1() const : Player *
+getPlayer2() const : Player *
+getPlayer3() const : Player *
+getPlayer4() const : Player *
+getBank() const : Bank *
+getCurrentPlayer() const : Player *
+get_players() : QVector<Player> &
+setPlayer1(player : Player) : void
+setPlayer2(player : Player) : void
+setPlayer3(player : Player) : void
+setPlayer4(player : Player) : void
+setBank(bank : Bank) : void
+setCurrentPlayer(currentPlayer : Player) : void
+currentPlayer() const : int
+winPlayer1() : bool
+winPlayer2() : bool
+winPlayer3() : bool
+winPlayer4() : bool
+BuildHouse() : void
+BuildCity() : void
+BuildRoad() : void
+win() : bool
+setWin() : bool : void
+TurnResult : int, board : Board * : void
+ChangeCurrentPlayer() : void
+nextPlayer() : void
+set_can_build_house(value : bool) : void
+set_can_build_city(value : bool) : void
+get_can_build_house() const : bool
+get_can_build_city() const : bool
+can_build_house() : bool
+can_build_city() : bool
+can_build_road() : bool
+can_get_magic_cards() : bool
+MagicCards() : void

```

```

Class Board
numOfNodes = 6 : const int
m_hasTrap : false : bool
m_trap : GUI_Node *
m_curr_color : QColor
m_road_color : QColor
m_first_turn = true : bool
m_nodes : QVector<Node> *
m_gui_nodes : QVector<Node> *
m_fields : QVector<Field> *
m_setHouse = false : bool
m_setCity = false : bool
m_setRoad = false : bool
+Board()
+Board(parent : QWidget * parent) : void
+Board()
+addNode(Node * node) : void
+addTrap(Trap * trap) : void
+mousePressEvent : QGraphicsSceneMouseEvent * override : void
+setHasTrap() : bool : void
+setCurrColor(color : QColor) : void
+getCurrColor() : QColor
+setRoadColor(color : QColor) : void
+getRoadColor() : QColor
+first_turn() const : bool
+set_first_turn(value : bool) : void
+manageResourceHouse() : void -> signal
+manageResourceCity() : void -> signal
+manageResourceRoad() : void -> signal
+setHasTrap() : void -> slot
+setHouse(x : bool) : void -> slot

```

```

Class Field
m_id : int
m_number : int
m_res_type : ResourceType
m_corners : QVector<Node>
m_up_corner : Node *
m_down_corner : Node *
m_bottom_left_corner : Node *
m_bottom_right_corner : Node *
m_up_left_corner : Node *
m_up_right_corner : Node *
m_edges : QVector<Road> *
m_right_edge : Road *
m_left_edge : Road *
m_bottom_right_edge : Road *
m_bottom_left_edge : Road *
m_up_right_edge : Road *
m_up_left_edge : Road *
m_is_robbet_on_the_field : bool
+Field(id : int, number : int, up_left_corner : Node *, up_corner : Node *, up_right_corner : Node *, bottom_right_corner : Node *, bottom_corner : Node *, bottom_left_corner : Node *)
+Field(const Field &) = delete
+Field(const Field &) : void
+get_id() const : int
+get_number() const : int
+get_res_type() const : ResourceType
+is_robbet_on_the_field() const : bool
+get_corners() : QVector<Node> &
+get_edges() const : QVector<Road> *
+get_robbet_on_the_field() const : bool
+get_up_corner() const : Node *
+get_down_corner() const : Node *
+get_bottom_right_corner() const : Node *
+get_bottom_left_corner() const : Node *
+get_up_left_corner() const : Node *
+get_up_right_corner() const : Node *
+get_right_edge() const : Road *
+get_left_edge() const : Road *
+get_bottom_right_edge() : Road *
+get_bottom_left_edge() : Road *
+get_up_right_edge() : Road *
+get_up_left_edge() : Road *
+set_res_type(res_type : ResourceType) : void

```

```

Class Dice
m_dice1 : int
m_dice2 : int
m_button_is_clicked : bool
m_dice_is_rolled : bool
+Dice()
+Dice(const Dice &) = delete
+set_button_is_clicked(button_clicked : bool) : void
+set_dice1(num : int) : void
+set_dice2(num : int) : void
+set_dice_is_rolled(value : bool) : void
+get_dice1() const : int
+get_dice2() const : int
+get_button_clicked() const : bool
+get_dice_is_rolled() const : bool
+roll_dice() : void

```

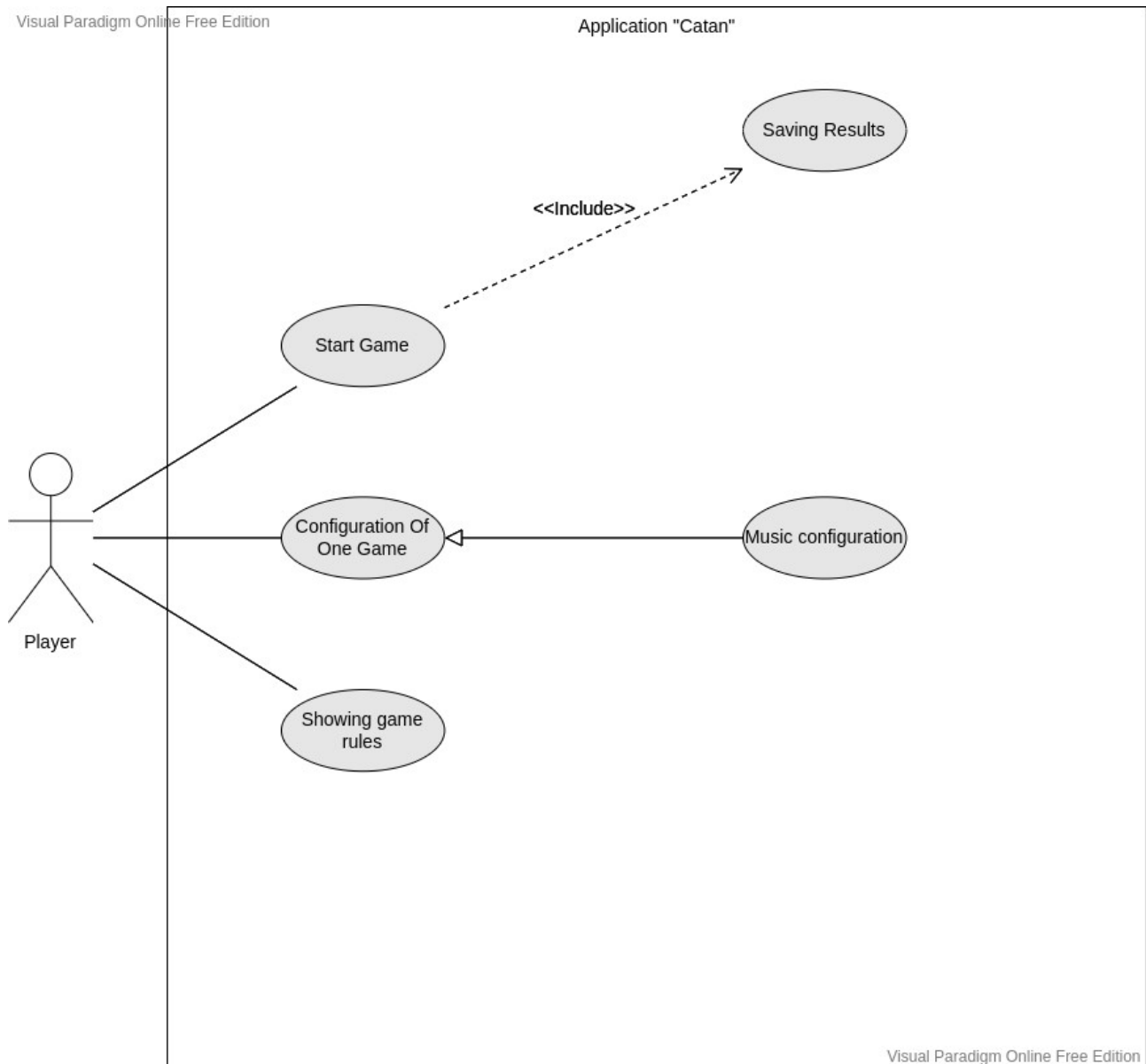
```

Class MainWindow
ui : Ui::MainWindow *
m_board : Board *
m_dice : Dice *
m_magic : QMediaPlayer *
m_game : Game *
+displayResource() : void
+displayBankResources() : void
+displayPlayerPoints() : void
+displayPlayerRoads() : void
+addNewBankNode(Node : GUI_Node) : void -> signal
+AddHouse() : void -> signal
+on_pbBuildGame_clicked() : void -> slot
+on_pbSettings_clicked() : void -> slot
+on_pbBackToMenu_clicked() : void -> slot
+on_pbBackToMenu_2_clicked() : void -> slot
+on_pbHelp_clicked() : void -> slot
+on_pbExit_clicked() : void -> slot
+on_pbContinue_clicked() : void -> slot
+on_pbBackToMenu_3_clicked() : void -> slot
+on_pushButton_clicked() : void -> slot
+on_pushButton_clicked_2 : void -> slot
+on_pushButton_clicked_3 : void -> slot
+on_pushButton_clicked_4 : void -> slot
+on_pushButton_clicked_5 : void -> slot
+on_pushButton_clicked_6 : void -> slot
+on_pushButton_clicked_7 : void -> slot
+on_pushButton_clicked_8 : void -> slot
+on_pushButton_clicked_9 : void -> slot
+on_pushButton_clicked_10 : void -> slot
+on_pushButton_clicked_11 : void -> slot
+on_pushButton_clicked_12 : void -> slot
+on_pushButton_clicked_13 : void -> slot
+on_pushButton_clicked_14 : void -> slot
+on_pushButton_clicked_15 : void -> slot
+on_pushButton_clicked_16 : void -> slot
+on_pushButton_clicked_17 : void -> slot
+on_pushButton_clicked_18 : void -> slot
+on_pushButton_clicked_19 : void -> slot
+on_pushButton_clicked_20 : void -> slot
+on_pushButton_clicked_21 : void -> slot
+on_pushButton_clicked_22 : void -> slot
+on_pushButton_clicked_23 : void -> slot
+on_pushButton_clicked_24 : void -> slot
+on_pushButton_clicked_25 : void -> slot
+on_pushButton_clicked_26 : void -> slot
+on_pushButton_clicked_27 : void -> slot
+on_pushButton_clicked_28 : void -> slot
+on_pushButton_clicked_29 : void -> slot
+on_pushButton_clicked_30 : void -> slot
+on_pushButton_clicked_31 : void -> slot
+on_pushButton_clicked_32 : void -> slot
+on_pushButton_clicked_33 : void -> slot
+on_pushButton_clicked_34 : void -> slot
+on_pushButton_clicked_35 : void -> slot
+on_pushButton_clicked_36 : void -> slot
+on_pushButton_clicked_37 : void -> slot
+on_pushButton_clicked_38 : void -> slot
+on_pushButton_clicked_39 : void -> slot
+on_pushButton_clicked_40 : void -> slot
+on_pushButton_clicked_41 : void -> slot
+on_pushButton_clicked_42 : void -> slot
+on_pushButton_clicked_43 : void -> slot
+on_pushButton_clicked_44 : void -> slot
+on_pushButton_clicked_45 : void -> slot
+on_pushButton_clicked_46 : void -> slot
+on_pushButton_clicked_47 : void -> slot
+on_pushButton_clicked_48 : void -> slot
+on_pushButton_clicked_49 : void -> slot
+on_pushButton_clicked_50 : void -> slot
+on_pushButton_clicked_51 : void -> slot
+on_pushButton_clicked_52 : void -> slot
+on_pushButton_clicked_53 : void -> slot
+on_pushButton_clicked_54 : void -> slot
+on_pushButton_clicked_55 : void -> slot
+on_pushButton_clicked_56 : void -> slot
+on_pushButton_clicked_57 : void -> slot
+on_pushButton_clicked_58 : void -> slot
+on_pushButton_clicked_59 : void -> slot
+on_pushButton_clicked_60 : void -> slot
+on_pushButton_clicked_61 : void -> slot
+on_pushButton_clicked_62 : void -> slot
+on_pushButton_clicked_63 : void -> slot
+on_pushButton_clicked_64 : void -> slot
+on_pushButton_clicked_65 : void -> slot
+on_pushButton_clicked_66 : void -> slot
+on_pushButton_clicked_67 : void -> slot
+on_pushButton_clicked_68 : void -> slot
+on_pushButton_clicked_69 : void -> slot
+on_pushButton_clicked_70 : void -> slot
+on_pushButton_clicked_71 : void -> slot
+on_pushButton_clicked_72 : void -> slot
+on_pushButton_clicked_73 : void -> slot
+on_pushButton_clicked_74 : void -> slot
+on_pushButton_clicked_75 : void -> slot
+on_pushButton_clicked_76 : void -> slot
+on_pushButton_clicked_77 : void -> slot
+on_pushButton_clicked_78 : void -> slot
+on_pushButton_clicked_79 : void -> slot
+on_pushButton_clicked_80 : void -> slot
+on_pushButton_clicked_81 : void -> slot
+on_pushButton_clicked_82 : void -> slot
+on_pushButton_clicked_83 : void -> slot
+on_pushButton_clicked_84 : void -> slot
+on_pushButton_clicked_85 : void -> slot
+on_pushButton_clicked_86 : void -> slot
+on_pushButton_clicked_87 : void -> slot
+on_pushButton_clicked_88 : void -> slot
+on_pushButton_clicked_89 : void -> slot
+on_pushButton_clicked_90 : void -> slot
+on_pushButton_clicked_91 : void -> slot
+on_pushButton_clicked_92 : void -> slot
+on_pushButton_clicked_93 : void -> slot
+on_pushButton_clicked_94 : void -> slot
+on_pushButton_clicked_95 : void -> slot
+on_pushButton_clicked_96 : void -> slot
+on_pushButton_clicked_97 : void -> slot
+on_pushButton_clicked_98 : void -> slot
+on_pushButton_clicked_99 : void -> slot
+on_pushButton_clicked_100 : void -> slot

```

2. Dijagram slucajeva upotrebe

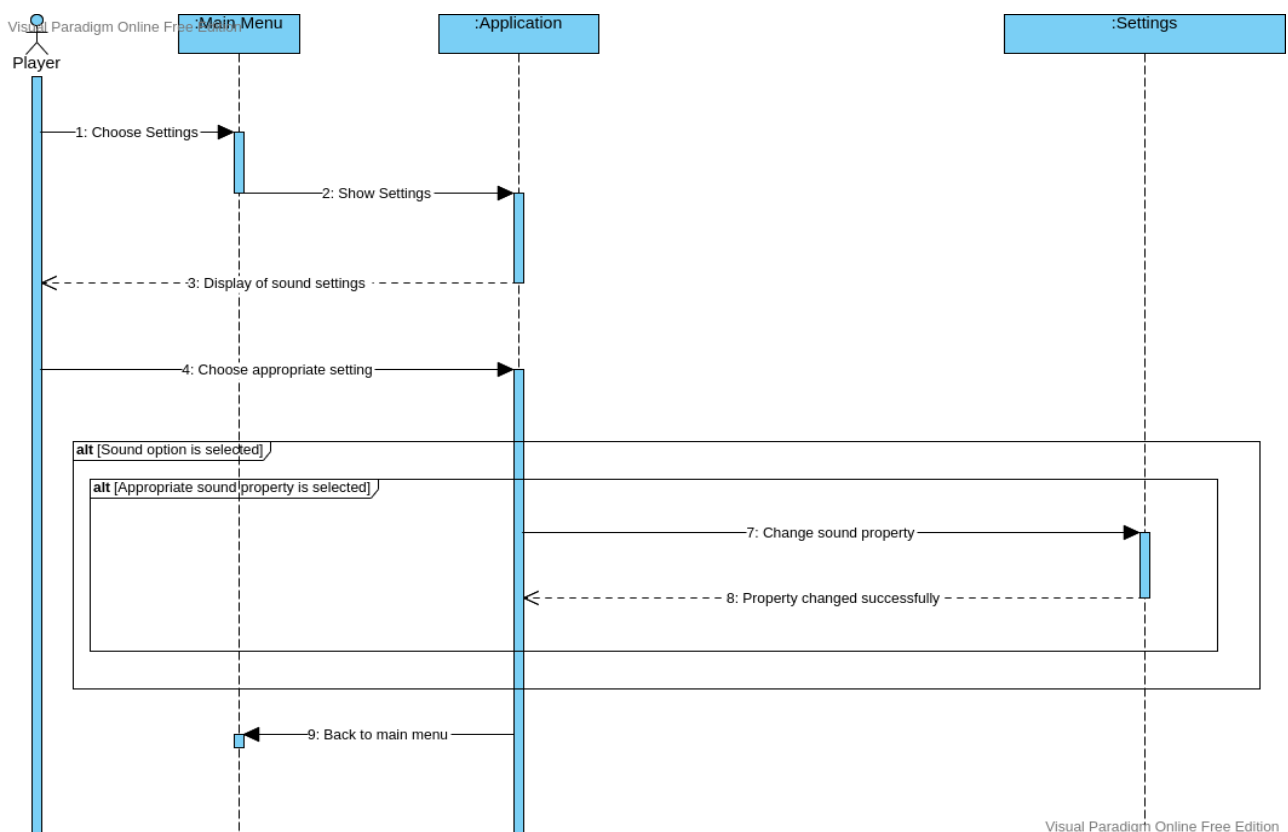
-Igrac na raspolaganju ima mogucnost da izmeni podesavanja, zapocne igru ili prikaze pravila igre. Nakon zavrsetka partije prikazuje se ime pobednika.



3. Dijagra za podesavanja igre

-Prilikom odabira dugmeta „Settings“ u pocetnom meniju, igracu se prikazuje novi prozor u kome moze da podesava zvuk i brzinu tokom partije.

- U okviru sekcije „Music“ igrac moze da bira da li ce svoju partiju obogatiti predvidjenom muzikom za ovu igricu (pritiskom dugmeta „ON“) ili ce istu igrati bez ikakvih zvucnik efekata (pritiskom dugmeta „OFF“)
- Podrazumevani izbor za ovo podesavanje je „ON“.
- U okviru sekcije „Game speed“ igrac moze da bira da li ce svoju partiju igrati klasicnom predvidjenom brzinom za igrace (pritiskom radio buttona „SLOW“) ili ce istu igrati ubrzano, tj. Sa skracenim intervalima tajmera (pritiskom radio buttona „FAST“).
- Podrazumevani izbor za ovo podesavanje je „SLOW“.
- Nakon sto odabere podesavanja koja mu odgovaraju, igrac se vraca u pocetni meni pritiskom na dugme „Back to main menu“, odabrana podesavanja se dalje prosledjuju aplikaciji i opcije podesavanja su uspesno sacuvana.

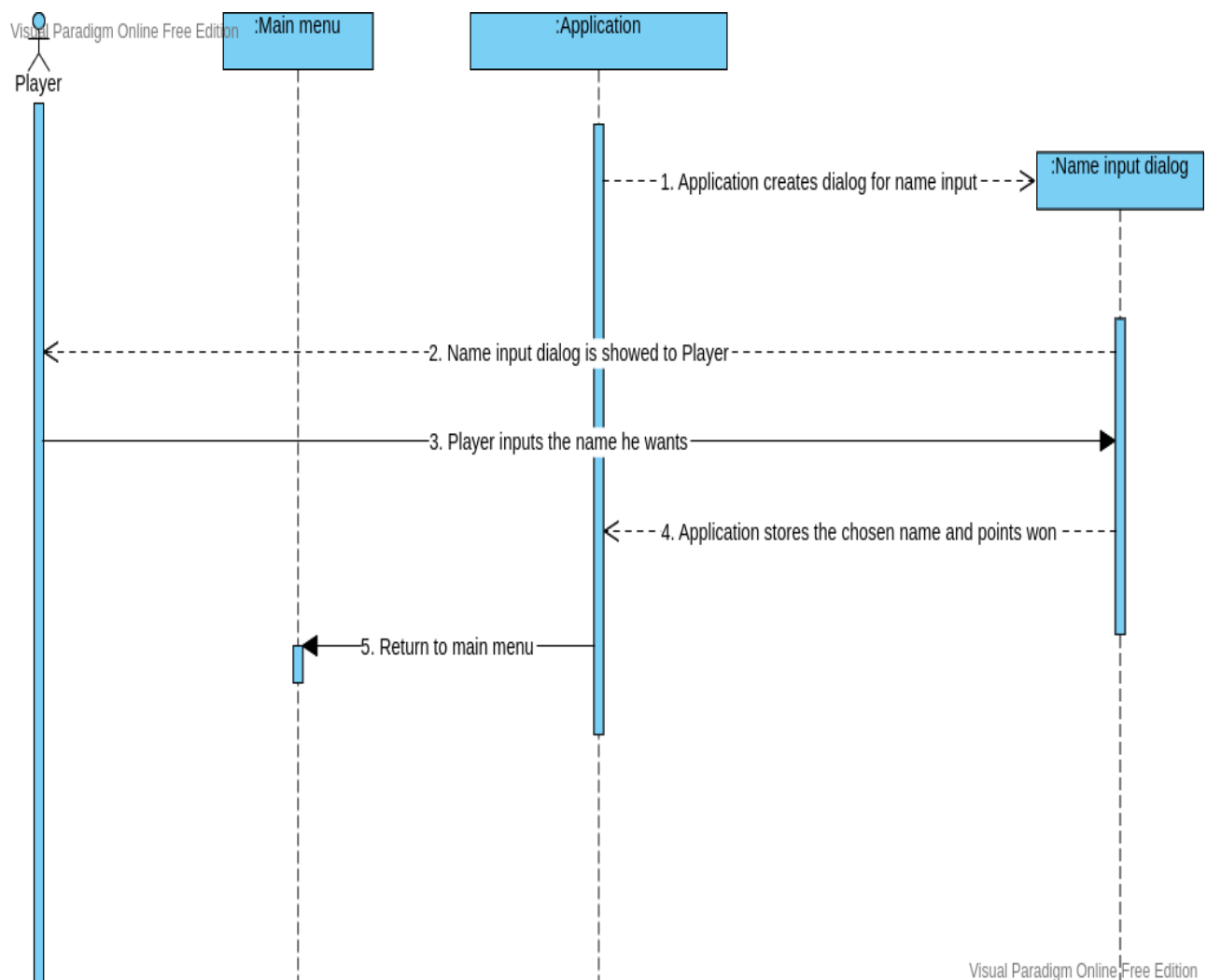


4. Dijagram konacnih rezultata

-Prilikom odabira dugmeta „Start Game“ u pocetnom meniju, igracu se prikazuje novi prozor u kojem pored boje koja ce ga predstavljati unosi i svoje ime koje ce biti vidljivo tokom partije.

-Nakon sto neki od igraca dostigne 10 poena, konacni rezultati svakog igraca se cuvaju, a igracima se prikazuje prozor u kojem se pored pobednika prikazuje i njihov ostvareni plasman.

-Nakon prikazanih rezultata igrica nas vraca u pocetni meni.

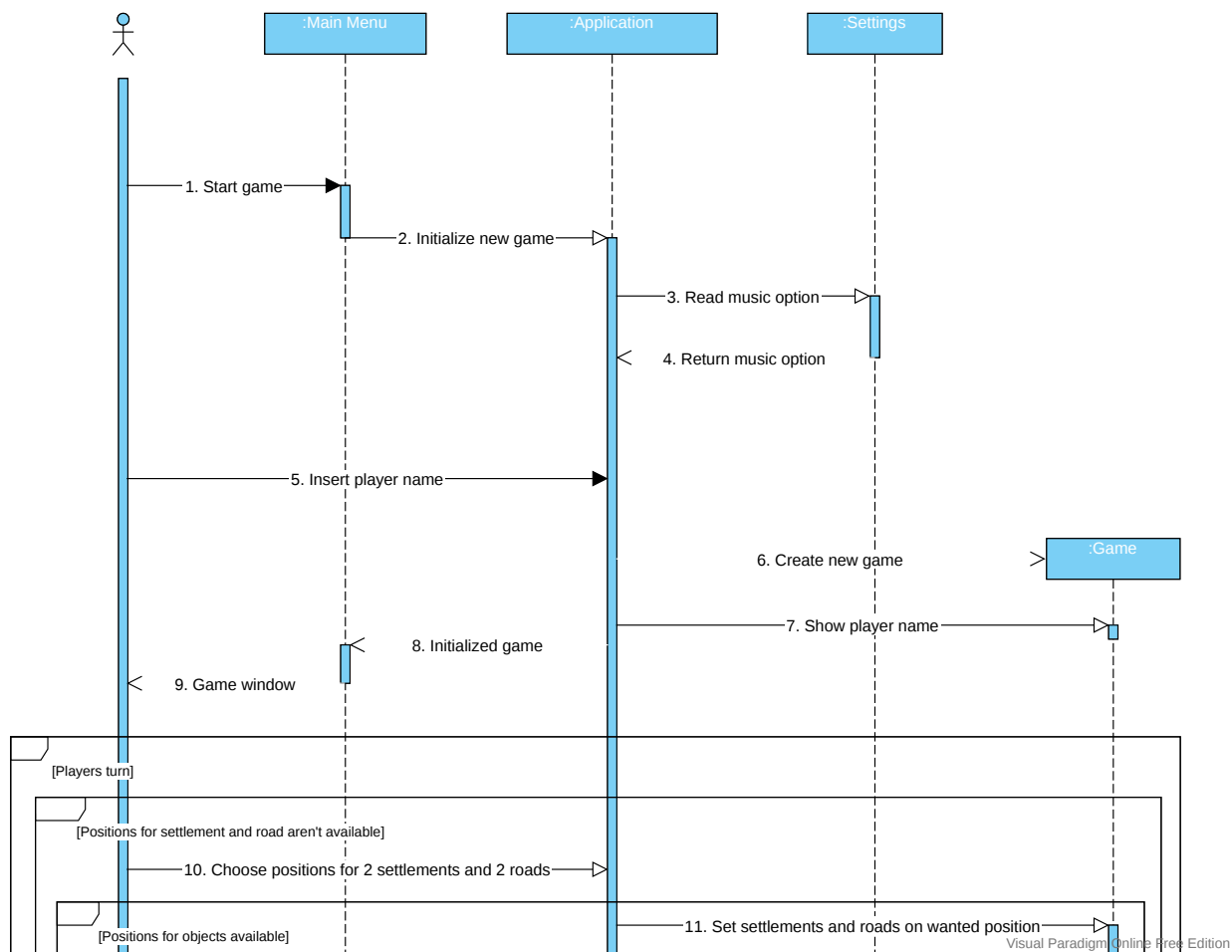


5. Dijagram sekvence – One Game

- Dijagram sekvence za jednu partiju nam pokazuje tok partije kroz glavne tacke:

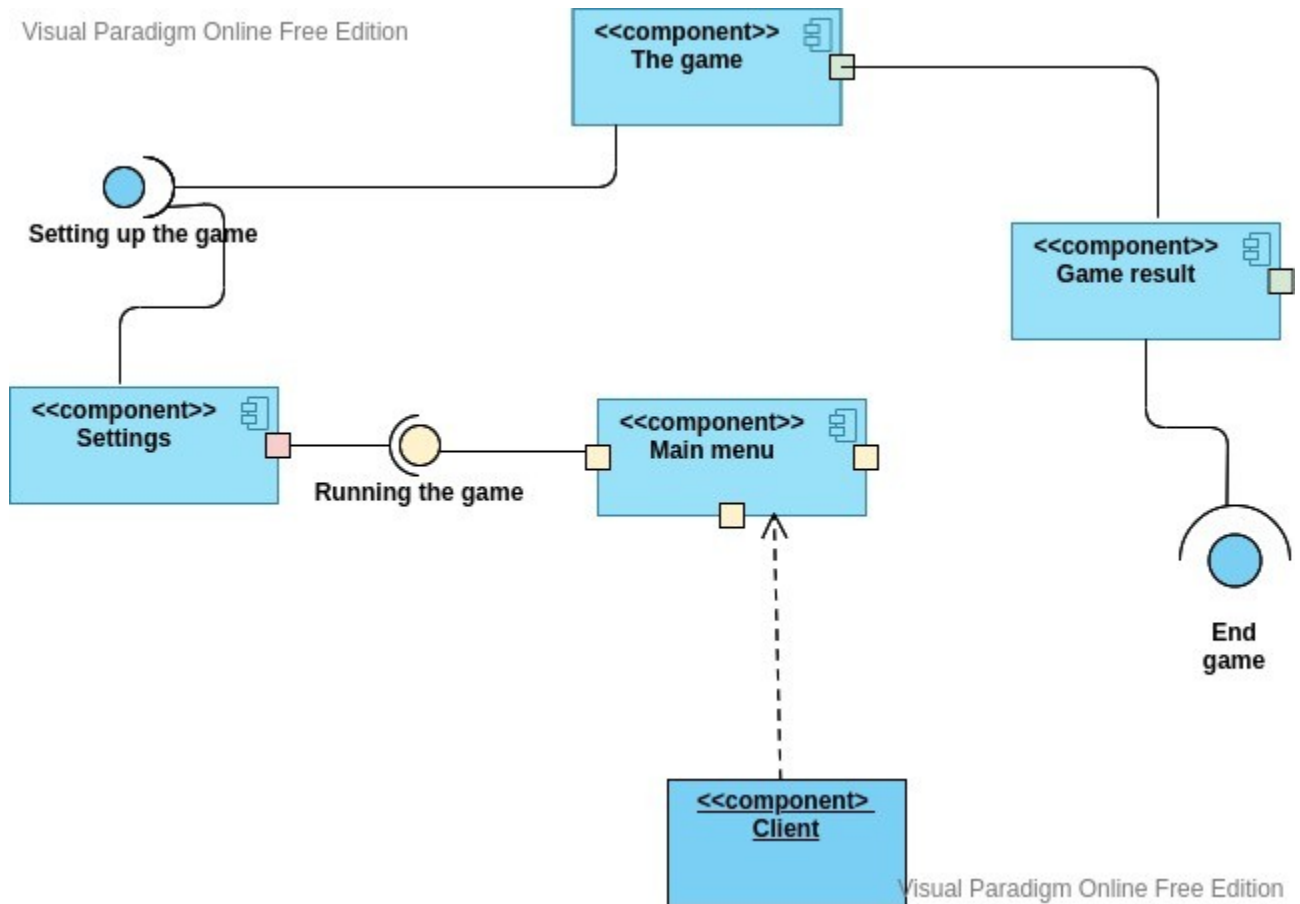
- ulazak u Main menu
- startovanje aplikacije
- namestanje podesavanja
- odigravanje partije
- ispis konacnih rezultata

Visual Paradigm Online Free Edition



6. Dijagram komponenti

Dijagram komponenata prikazuje organizaciju i zavisnosti između komponenata.



Opis slucajeva upotrebe:

- Kratak opis igre: Igrac zapocinje jednu partiju igre iz glavnog menija aplikacije. Aplikacija ucitava podesavanja partije kao i asete potrebne za odigravanje jedne partije. Nakon odigrane partije aplikacija skladišti podatke o odigranoj partiji.

• Akteri:

- Igrac

- Bot

- Preduslovi: Aplikacija je pokrenuta i prikazuje glavni meni
- Postuslovi: Informacije o odigranoj partiji se trajno cuvaju
- Osnovni tok:

1. Igrac bira dugme "Zapocni igru" iz glavnog menija

- 1.1.1. Potrebno je da se ostala 3 igraca prikljuce partiji

- 1.1.2. Ukoliko se u roku od 2 minuta nisu ostala 3 igraca priklucila partiji, aplikacija se vraca na pocetni meni

- 1.1.3. Ukoliko su se svi igraci prikljucili partiji prelazi se na korak 2.

2. Aplikacija dohvata informacije o podesavanjima partije.

3. Aplikacija konstruise novu partiju i zapocinje je.

4. Svaki od igraca baca 2 kockice, na osnovu cega se odlucuje redosled igraca koji su na potezu, pri cemu najvecu prednost ima igrac sa najvećim zbirom dobijenih brojeva na kockicama

- 4.1 U slucaju da su 2 ili vise igraca dobila isti broj pri bacanju kockica, ponavlja se bacanje izmedju tih igraca iznova dok god se ne ustanovi redosled igranja, pridrzavajući se rasporeda odredjenog prilikom prethodnog bacanja

5. Postavljanje jednog naselja i jednog puta koji je vezan za to isto naselje u polja koja nisu vec zauzeta u narednom redosledu:

- 5.1 Na osnovu redosleda igraca utvrdjenog u koraku 4. igraci izvrsavaju akciju opisanu u koraku 5.

- 5.1.1 Uslov za postavljanje naselja na odredjeno polje jeste da ne postoji nijedno drugo naselje na bilo kom od susednih polja

5.1.1.1 Polja su susedna ukoliko ih razdvaja samo jedan putic

5.2 Obrnutim redosledom u odnosu na korak 5.1 svaki igrac izvorsava akciju opisanu u koraku 5. pri cemu u zavisnosti od mesta postavljanja naselja dobija po jednu karticu od svakog resursa koji je pokriven tim naseljem

6. Sve dok neki igrac ne dostigne 10 poena ponavljaju se sledeci koraci redosledom utvrdjenim u koraku 5.:

6.1 Igrac ima 10 sekundi za bacanje dve kockice i u zavisnosti od zbira brojeva na kocikcama se postupa na sledeci nacin:

6.1.1. Ukoliko je dobijeni zbir na kockicama jednak broju 7 postupa se po sledecim pravilima

6.1.1.1. Svaki od igraca koji u trenutku desavanja dogadjaja opisanog u koraku 6.1.1. koji kod sebe ima ukupno vise od 7 karata bilo kog od resursa mora da oslobodi proizvoljnih pola resursa nazad u banku

6.1.1.2. Igrac koji je bacao kockice u dogadjaju opisanom u koraku 6.1.1. dobija posebnu sposobnost u vidu pijuna

6.1.1.3 Postavljanjem pijuna na neko od polja igrac ostvaruje pravo na oduzimanje jednog od resursa po slucajnom izboru igracu koji je to polje zauzeo

6.1.1.3.1. Ukoliko je jedno polje zauzeto od strane vise igraca, igrac bira od kog igraca ce resurs biti oduzet

6.1.2. Ukoliko je dobijeni zbir na kockicama razlicit od broja 7 postupa se po sledecim pravilima

6.1.2.1 Ukoliko igrac ima izgradjenu kucicu koja pokriva neko polje na kome se nalazi broj jednak dobijenom zbiru kockica iz koraka 6.1 dobija jednu karticu resursa na kome se nalazi taj broj

6.1.2.2 Ukoliko igrac ima izgradjen grad koji pokriva neko polje na kome se nalazi broj jednak dobijenom zbiru kockica iz koraka 6.1 dobija dve kartice resursa na kome se nalazi taj broj

6.2 Svaki igrac ima 2 minuta na raspolaganju za odigravanje svog poteza

6.3 U okviru svog poteza svaki igrac moze izgraditi neki objekat ili kupiti magicnu karticu pod sledecim uslovima:

6.3.1 Da bi igrac sagradio jedan putic potrebna mu je jedna kartica resursa gline i jedna kartica resursa drveta

6.3.2 Da bi igrac sagradio jedno naselje potrebna mu je jedna kartica resursa sena, jedna kratka kartica resursa gline, jedna kartica resursa drveta i jedna kartica resursa ovce

6.3.3 Da bi igrac sagradio jedan grad potrebno mu je da ima tri kartice resursa kamena i dve kartice resursa sena takodje grad moze da bude sagradjen samo na mestime gde je taj igrac prethodno imao izgradjeno naselje

6.3.4 Da bi igrac dobio od banke jednu magicnu karticu potrebno je da

kod sebe ima jednu karticu resursa ovce, jednu karticu resursa sena i jednu karticu resursa kamena

6.3.5 Prilikom svake izgradje nekog objekta ili kupovine magicne kartice igrac vraca banci odredjeni broj resursa pod stavkama 6.3.1 , 6.3.2 , 6.3.3 , 6.3.4

6.4 U svakom potezu svaki igrac ima pravo da banci vrati, ukoliko ima mogucnost, 4 kartice istog resursa i da uzme iz banke jednu karticu resursa po zelji

6.5 Potez svakog igraca se zavrшава ukoliko pritisne dugme "Dalje" ili istekne vreme predvidjeno za svaki potez i tada se prelazi na sledeceg igraca po rasporedu utvrdjenom u koraku 4.

7. Poeni se dobijaju na sledeci nacin:

7.1 Za svako izgradjeno naselje igrac dobija jedan poena

7.2 Za svaki izgradjeni grad igrac dobija dva poena ali gubi jedan poena jer se grad gradi na mestu nekog od naselja tog igraca

7.3 Ukoliko igrac ima vise od pet povezanih putica, i ako niko od ostalih igraca nema vise poveznih putica od njega, igrad dobija dva poena

7.3.1 Dva poena dobijena za najvise povezanih putica igrac moze izgubiti u slucaju da drugi igrac u medjuvremenu sagradi vise povezanih putica od njega

8. Na kraju svake partije informacije o odigranoj partiji se cuvaju.

9. Aplikacija prikazuje glavni meni.

- Alternativni tokovi:

– A1: Neočekivani izlaz iz aplikacije. U slučaju da u bilo kom koraku korisnik isključi aplikaciju, sve eventualno zapamćene informacije o trenutnoj partiji igre se poništavaju i aplikacija prestaje sa radom. Slučaj upotrebe se završava.

- Podtokovi: /

- Specijalni zahtevi: /

- Dodatne informacije: Tokom trajanja partije, aplikacija pamti informacije o broju poena svakog od igrača